

9.1 Introduction

In Chapter-2 you learned about the applications of Internet of Things for homes, cities, environment, energy systems, retail, logistics, industry, agriculture and health. This chapter provides concrete implementations of several of these applications helping you understand how sophisticated applications are designed and deployed. The case studies are based on the IoT design methodology described in Chapter-5. The IoT device used for the case studies is the Raspberry Pi mini-computer. The case studies are implemented in Python and use the Django framework. You learned about basics of Python and Django in Chapters 6 and 8, respectively. However, principles you have learned are not limited to these particular languages or platforms.

9.2 Home Automation

9.2.1 Smart Lighting

A design of a smart home automation system was described in Chapter-5 using the IoT design methodology. A concrete implementation of the system based on Django framework is described in this section. The purpose of the home automation system is to control the lights in a typical home remotely using a web application.

The system includes auto and manual modes. In auto mode, the system measures the light in a room and switches on the light when it gets dark. In manual mode, the system provides the option of manually and remotely switching on/off the light.

Figure 9.1 shows the deployment design of the home automation system. As explained in Chapter-5, the system has two REST services (mode and state) and a controller native service. Figures 9.2 and 9.3 show specifications of the mode and state REST services of the home automation system. The Mode service is a RESTful web service that sets mode to auto or manual (PUT request), or retrieves the current mode (GET request). The mode is updated to/retrieved from the database. The State service is a RESTful web service that sets the light appliance state to on/off (PUT request), or retrieves the current light state (GET request). The state is updated to/retrieved from the status database.

■ Box 9.1: Django model for mode and state REST services - models.py

```
from django.db import models

class Mode(models.Model):
    name = models.CharField(max_length=50)
```

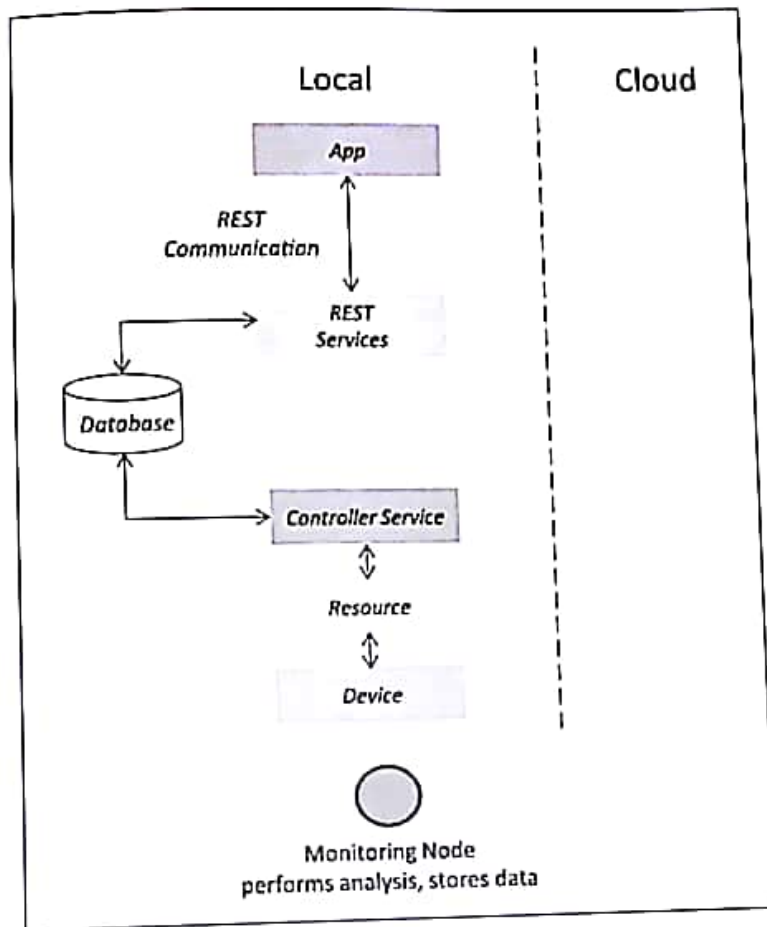


Figure 9.1: Deployment design of the home automation IoT system

```
class State(models.Model):
    name = models.CharField(max_length=50)
```

To start with the implementation of the system, we first map services to Django models. Box 9.1 shows the model fields for the REST services (state - on/off and mode - auto/manual).

■ Box 9.2: Serializers for mode and state REST services - serializers.py

```
from myapp.models import Mode, State
from rest_framework import serializers

class ModeSerializer(serializers.HyperlinkedModelSerializer):
```

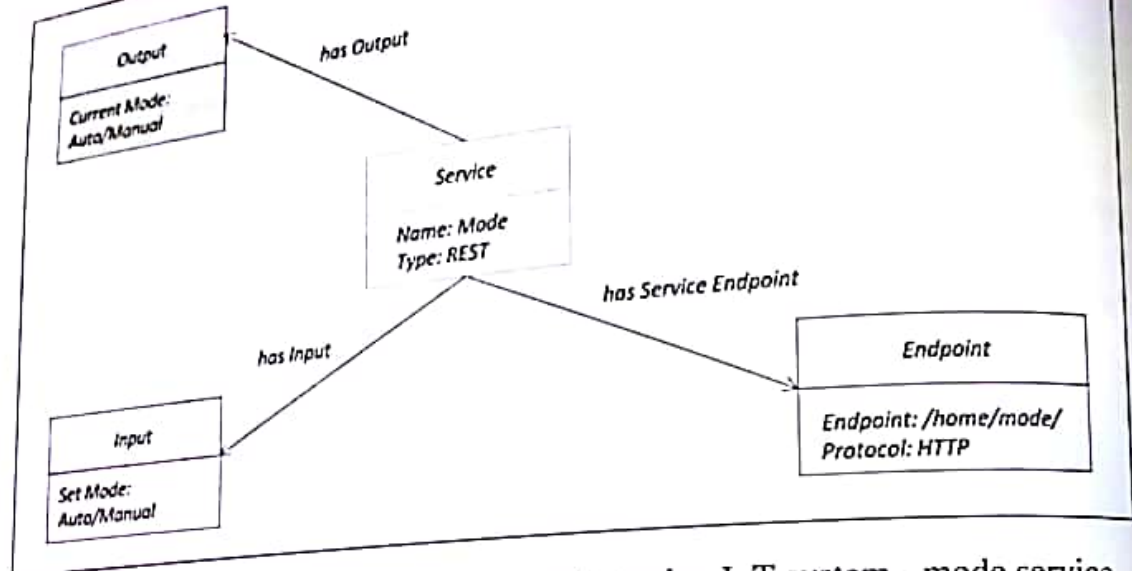


Figure 9.2: Service specification for home automation IoT system - mode service

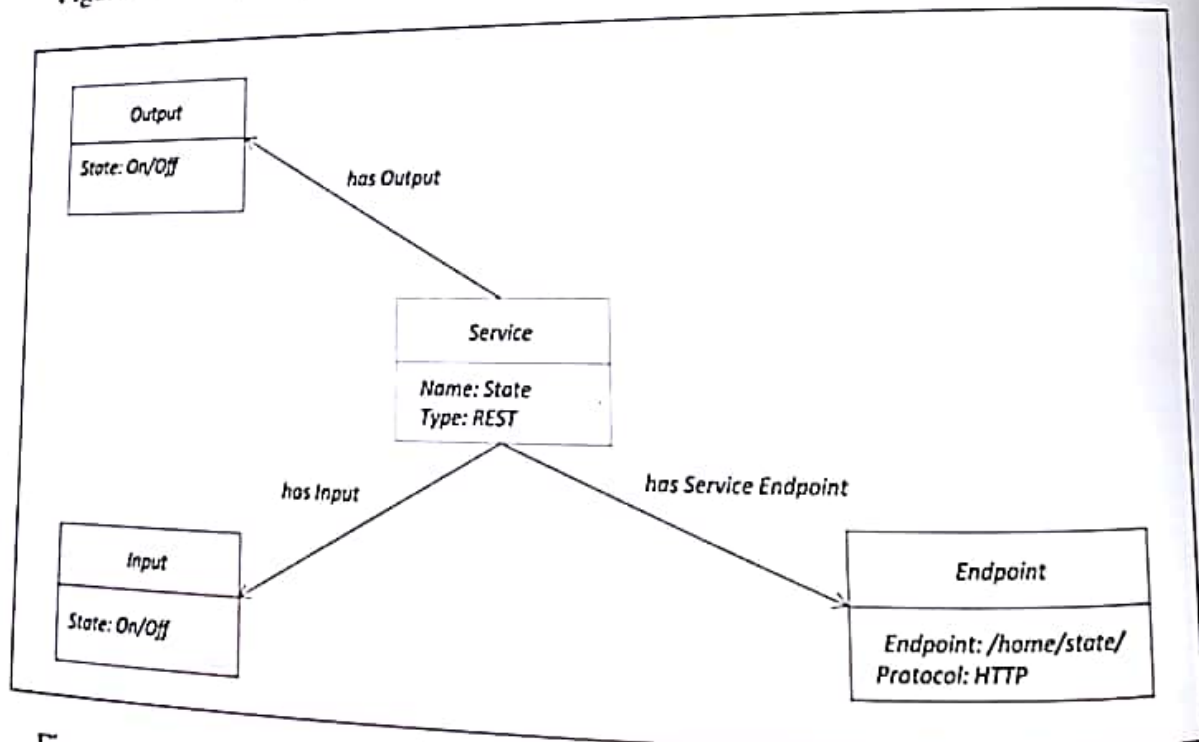


Figure 9.3: Service specification for home automation IoT system - state service

```
class Meta:
```

```

    model = Mode
    fields = ('url', 'name')

class StateSerializer(serializers.HyperlinkedModelSerializer):
    class Meta:
        model = State
        fields = ('url', 'name')

```

After implementing the Django model, we implement the model serializers. Serializers allow complex data (such as model instances) to be converted to native Python datatypes that can then be easily rendered into JSON, XML or other content types. Box 9.2 shows the serializers for mode and state REST services.

After implementing the serializers, we write ViewSets for the Django models. ViewSets combine the logic for a set of related views in a single class. Box 9.3 shows the Django views for REST services and home automation application. The ViewSets for the models (ModeViewSet and StateViewSet) are included in the views file. The application view (home) is described later in this section.

■ Box 9.3: Django views for REST services and home automation application - views.py

```

from myapp.models import Mode, State
from rest_framework import viewsets
from django.shortcuts import render_to_response
from django.template import RequestContext
from myapp.serializers import ModeSerializer, StateSerializer
import requests
import json

class ModeViewSet(viewsets.ModelViewSet):
    queryset = Mode.objects.all()
    serializer_class = ModeSerializer

class StateViewSet(viewsets.ModelViewSet):
    queryset = State.objects.all()
    serializer_class = StateSerializer

def home(request):
    out=""
    currentmode='auto'
    currentstate='off'

```

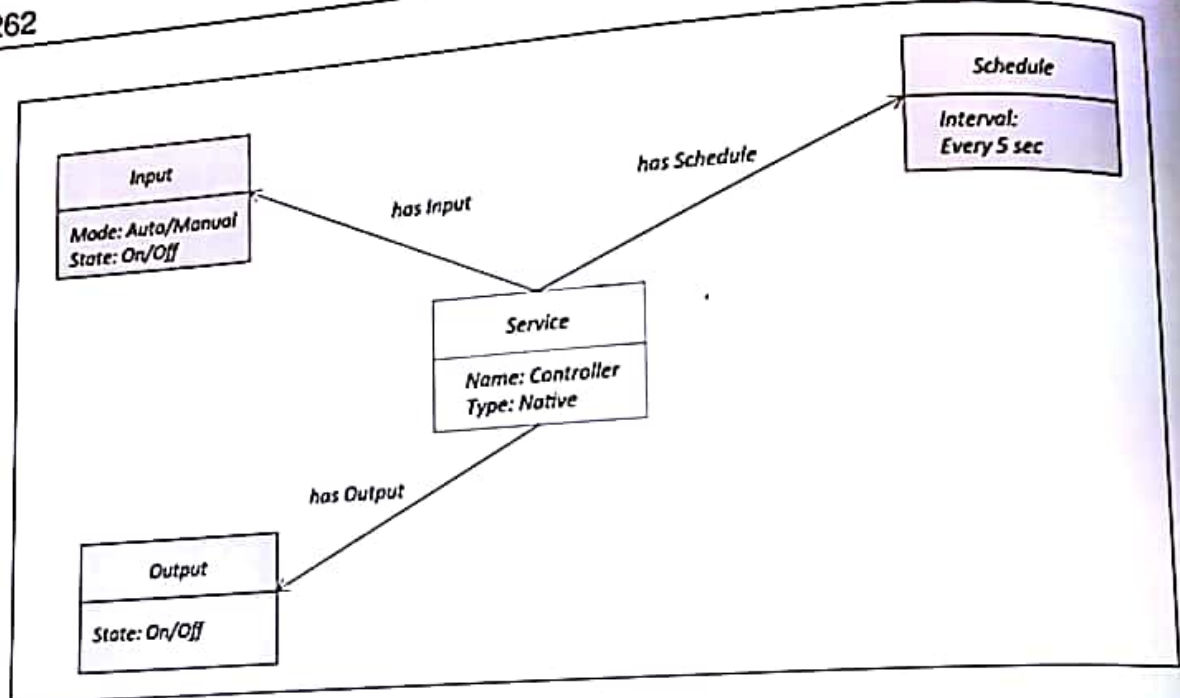


Figure 9.6: Controller service of the home automation IoT system

```

# Open SPI bus
spi = spidev.SpiDev()
spi.open(0,0)

#Light Level Threshold
threshold=200

#Function to read LDR connected to MCP3008
def readLDR():
    light_level = ReadChannel(LIGHT_CHANNEL)
    lux = ConvertLux(light_level,2)
    return lux

#Function to convert LDR reading to Lux
def ConvertLux(data,places):
    R=10 #10k-ohm resistor connected to LDR
    volts = (data * 3.3) / 1023
    volts = round(volts,places)
    lux=500*(3.3-volts)/(R*volts)
    return lux
  
```

```

        switchOnLight (LIGHT_PIN)
    else:
        switchOffLight (LIGHT_PIN)

    print 'Manual'+ ' - '+getCurrentState()

#Controller main function
def runController():
    currentMode=getCurrentMode()
    if currentMode=='auto':
        runAutoMode()
    elif currentMode=='manual':
        runManualMode()

    return true

while True:
    runController()
    time.sleep(5)

```

9.2.2 Home Intrusion Detection

You got an overview of home intrusion detection systems in Chapter-2. A concrete implementation of a home intrusion detection system is described in this section. The purpose of the home intrusion detection system is to detect intrusions using sensors (such as PIR sensors and door sensors) and raise alerts, if necessary.

Figure 9.7 shows the process diagram for the home intrusion detection system. Each room in the home has a PIR motion sensor and each door has a door sensor. These sensors can detect motion or opening of doors. Each sensor is read at regular intervals and the motion detection or door opening events are stored and alerts are sent.

Figure 9.8 shows the domain model for the home intrusion detection system. The domain model includes physical entities for room and door and the corresponding virtual entities. The device in this example is a single-board mini computer which has PIR and door sensors attached to it. The domain model also includes the services involved in the system.

Figure 9.9 shows the information model for the home intrusion detection system. The information model defines the attributes of room and door virtual entities and their possible values. The room virtual entity has an attribute 'motion' and the door virtual entity has an attribute 'state'.

The next step is to define the service specifications for the system. The services are derived from the process specification and the information model. The system has three services - (1) a RESTful web service that retrieves the current state of a door from the

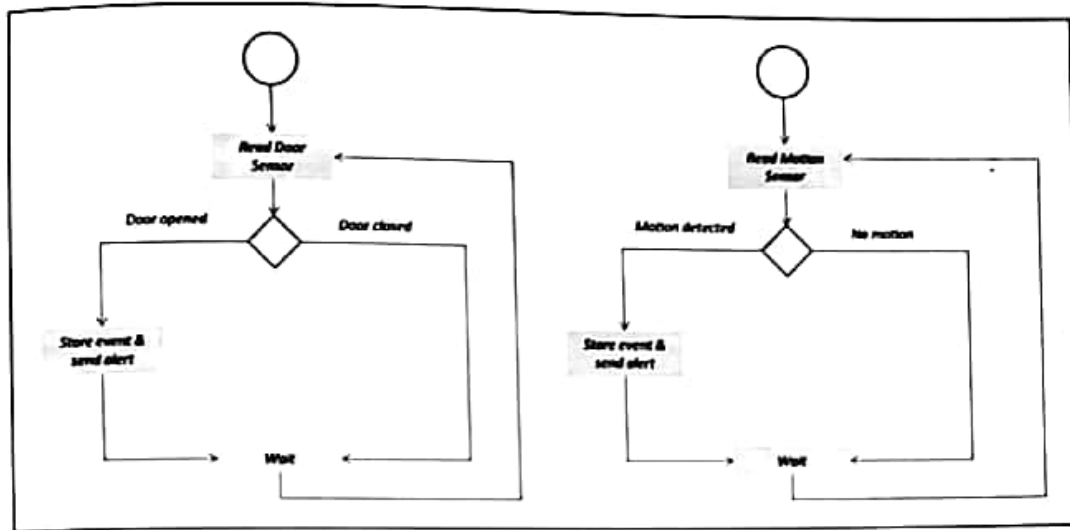


Figure 9.7: Process specification of the home intrusion detection IoT system

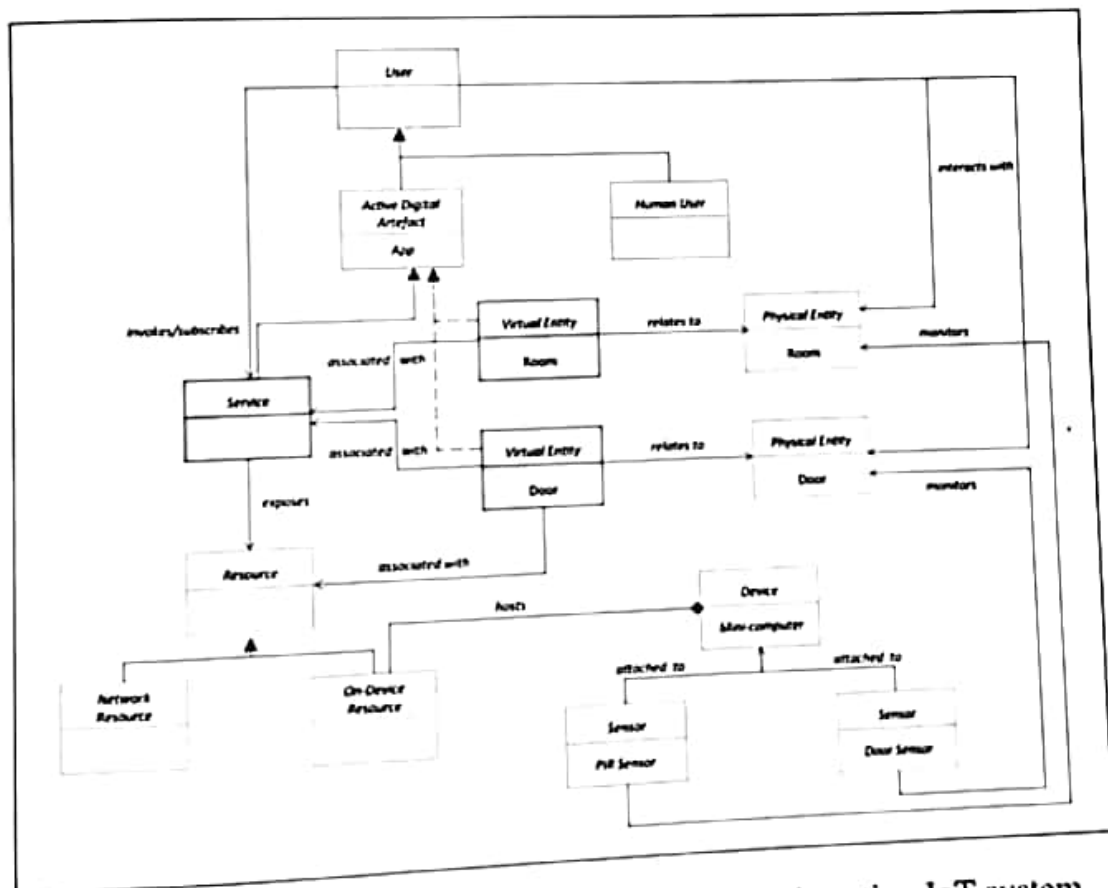


Figure 9.8: Domain model of the home intrusion detection IoT system

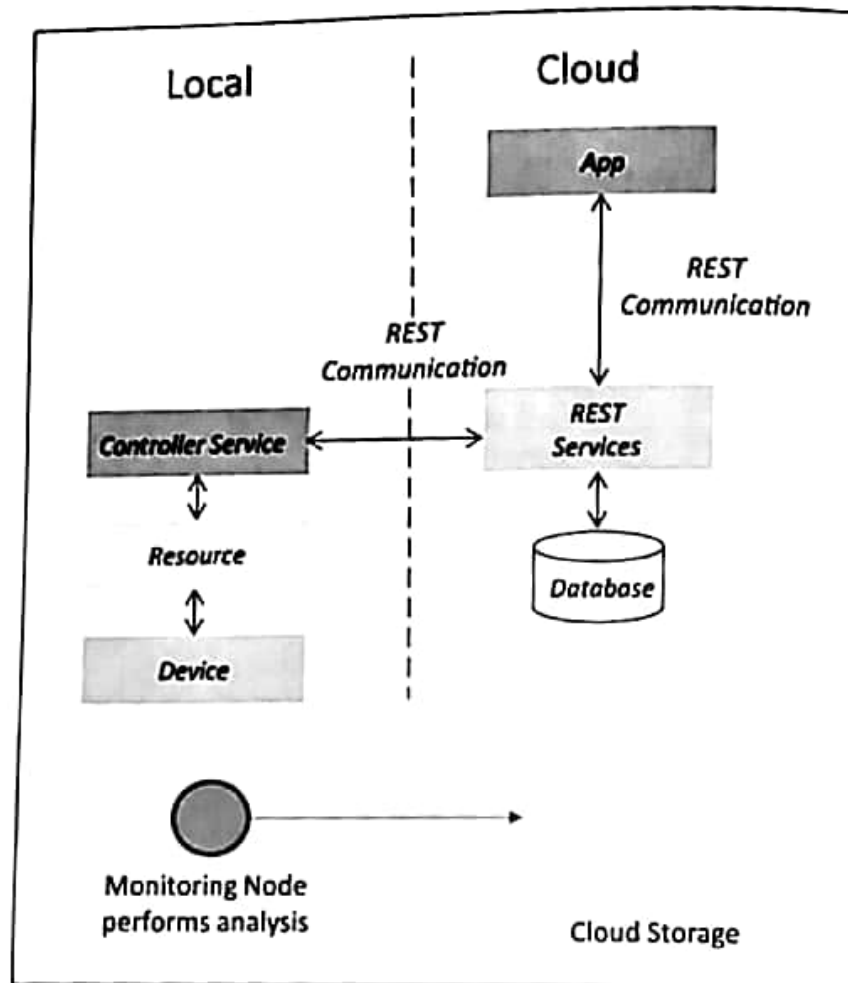


Figure 9.13: Deployment design for the home intrusion detection IoT system

The system uses Django framework for web application and REST service. The Django web application is backed by a MySQL database. The IoT device used for this example is Raspberry Pi along with the PIR and door sensors. Figure 9.15 shows a schematic diagram of the home intrusion detection system and Figure 9.16 shows how the sensors are deployed in a parking.

Let us now look at the implementation of the web application and services for the system. Box 9.7 shows the model fields for the room and door REST service. After implementing the Django model, we implement the model serializers that allows model instances to be converted to native Python datatypes. Box 9.8 shows the serializers for room and door REST services.

■ Box 9.7: Django model for room and door REST services - models.py

```
from django.db import models
```

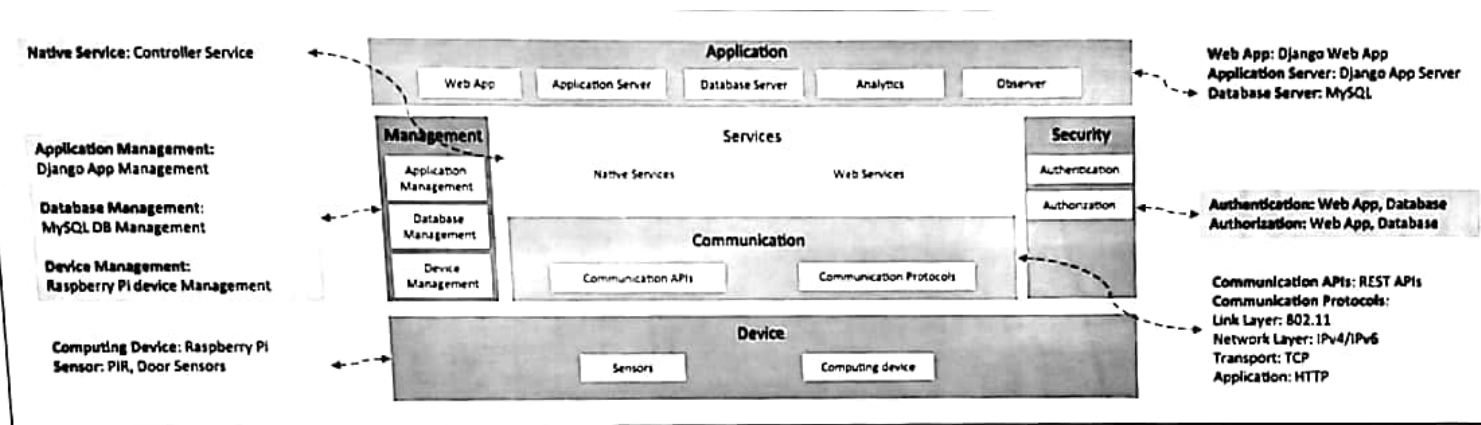


Figure 9.14: Functional & operational view specifications for home intrusion detection system

```

setPIRState('no')

doorState = readingDoorSensor()
if doorState == 1:
    setDoorState('open')
else:
    setDoorState('closed')

def setPIRState(val):
    values = {"state": val, "timestamp": str(time.time())}
    r=requests.put('http://127.0.0.1:8000/room/1/', data=values,
    auth=('username', 'password'))

def setDoorState(val):
    values = {"state": val, "timestamp": str(time.time())}
    r=requests.put('http://127.0.0.1:8000/door/1/', data=values,
    auth=('username', 'password'))

def setupController():
    r=requests.get('http://127.0.0.1:8000/config/1/', data=values,
    auth=('username', 'password'))
    configStr=r.text
    config = json.loads(configStr)
    global PIR_SENSOR_PIN = config['room1']
    global DOOR_SENSOR_PIN = config['door1']
    GPIO.setup(PIR_SENSOR_PIN,GPIO.IN)
    GPIO.setup(DOOR_SENSOR_PIN,GPIO.IN, pull_up_down=GPIO.PUD_UP)

setupController()
while True:
    runController()
    time.sleep(1)

```

9.3 Cities

9.3.1 Smart Parking

You got an overview of smart parking systems in Chapter-2. A concrete implementation of a smart parking IoT system is described in this section.

The purpose of a smart parking system is to detect the number of empty parking slots and send the information over the Internet to smart parking application backends. These applications can be accessed by drivers from smartphones, tablets or from in-car navigation systems. In smart parking, sensors are used for each parking slot, to detect whether the slot is empty or occupied. This information is aggregated by a local controller and then sent over

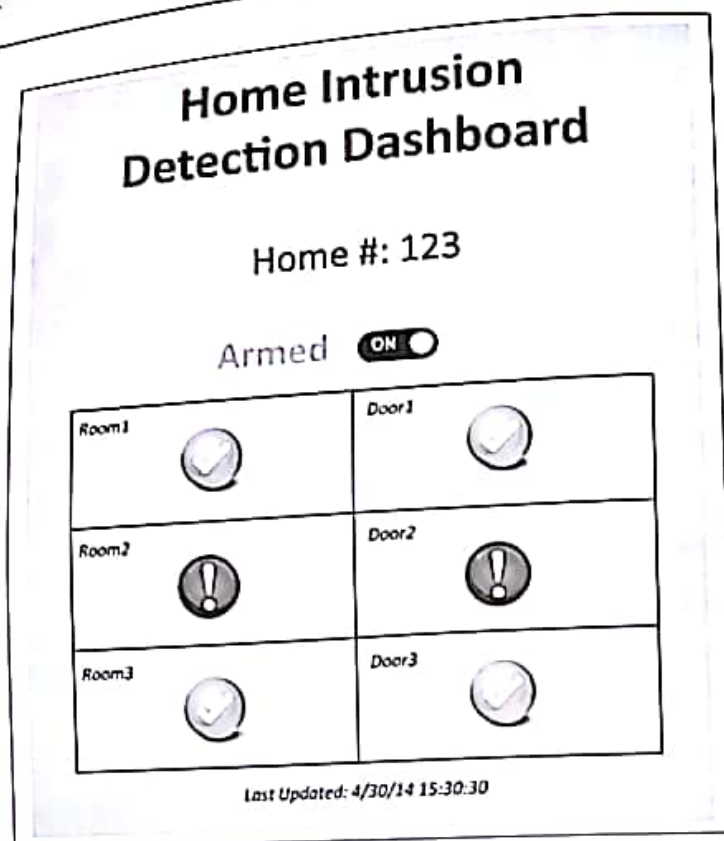


Figure 9.17: Home intrusion detection web application screenshot

the Internet to a server.

Figure 9.18 shows the process diagram for the smart parking system. Each parking slot has an ultrasonic sensor fixed above, which can detect the presence of a vehicle in the slot. Each sensor is read at regular intervals and the state of the parking slot (empty or occupied) is updated in a database.

Figure 9.19 shows the domain model for the smart parking system. The domain model includes a physical entity for the parking slot and the corresponding virtual entity. The device in this example is a single-board mini computer which has ultrasonic sensor attached to it. The domain model also includes the services involved in the system.

Figure 9.20 shows the information model for the smart parking system. The information model defines the attribute (state) of the parking slot virtual entity with two possible values (empty or occupied).

The next step is to define the service specifications for the system. The services are derived from the process specification and the information model. The smart parking system has two services - (1) a service that monitors the parking slots (using ultrasonic sensors) and

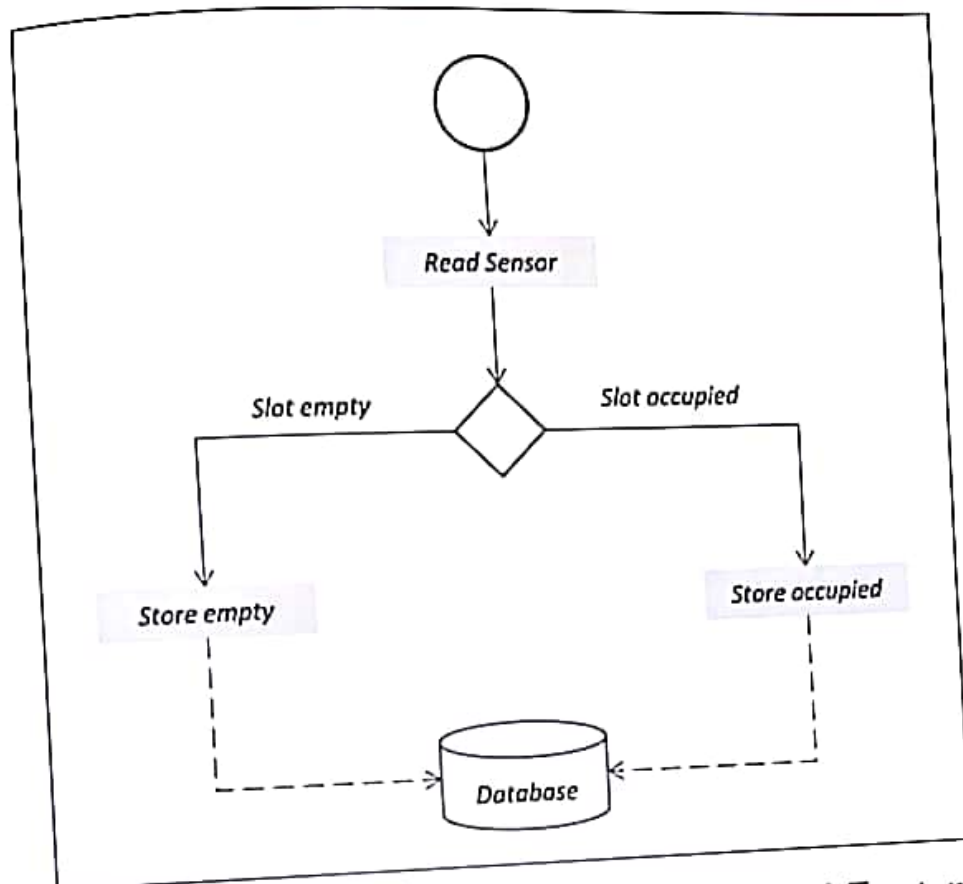


Figure 9.18: Process specification for the smart parking IoT system

updates the status in a database on the cloud (REST web service), (2) a service that retrieves the current state of the parking slots (controller native service). Figures 9.21 and 9.22 show specifications of the controller and state services of the smart parking system.

The functional view and the operational view specifications for smart parking system are similar to the specifications for the home intrusion detection system shown in Figure 9.14. The system uses Django framework for web application and REST service - both of which you learned about from earlier chapters of this book. The Django web application is supported by a MySQL database. The IoT device used for this example is Raspberry Pi along with the ultrasonic sensors. Figure 9.23 shows how the sensors are deployed in a parking and Figure 9.24 shows a schematic diagram of the smart parking system.

Let us now look at the implementation of the web application and services for the smart parking system. Box 9.13 shows the model fields for the state REST service. After implementing the Django model, we implement the model serializer which allows model

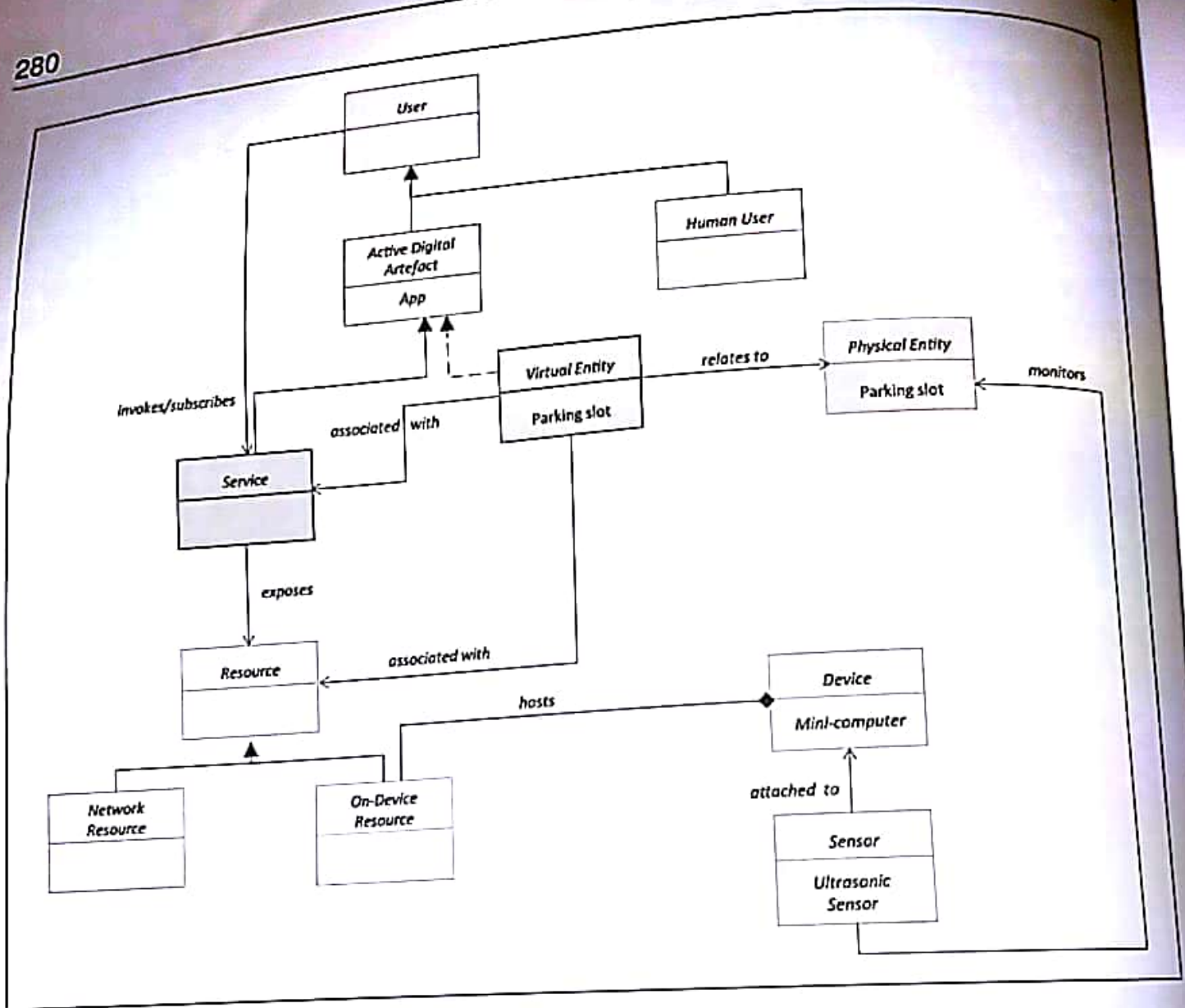


Figure 9.19: Domain model of the smart parking IoT system

instances to be converted to native Python datatypes. Box 9.14 shows the serializer for state REST service.

Box 9.13: Django model for REST service - models.py

```

from django.db import models

class State(models.Model):
    name = models.CharField(max_length=50)

```

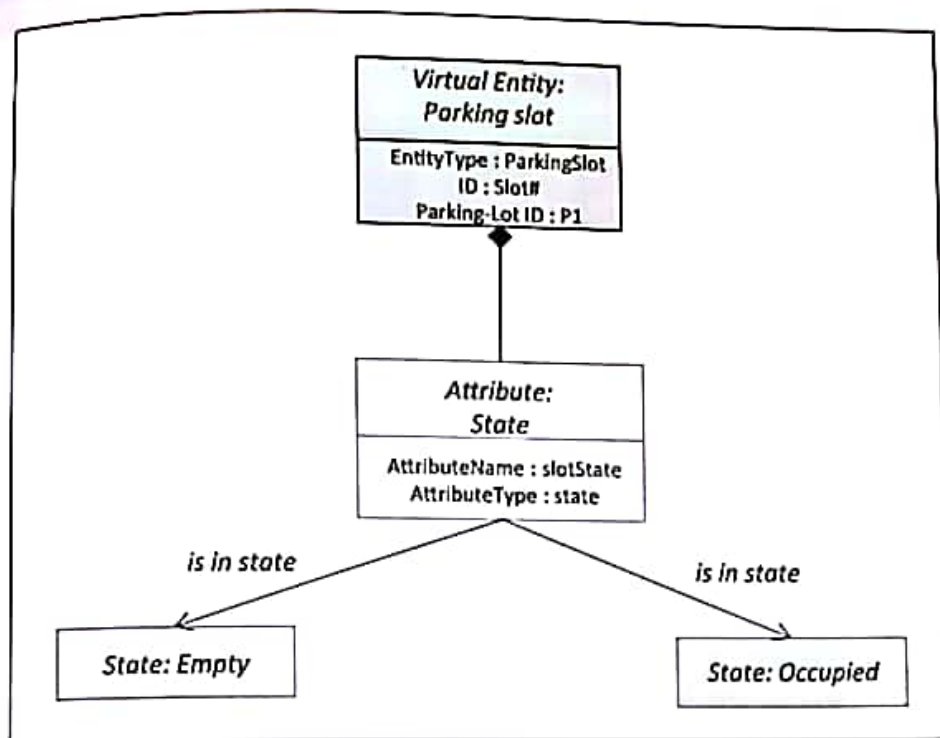


Figure 9.20: Information model of the smart parking IoT system

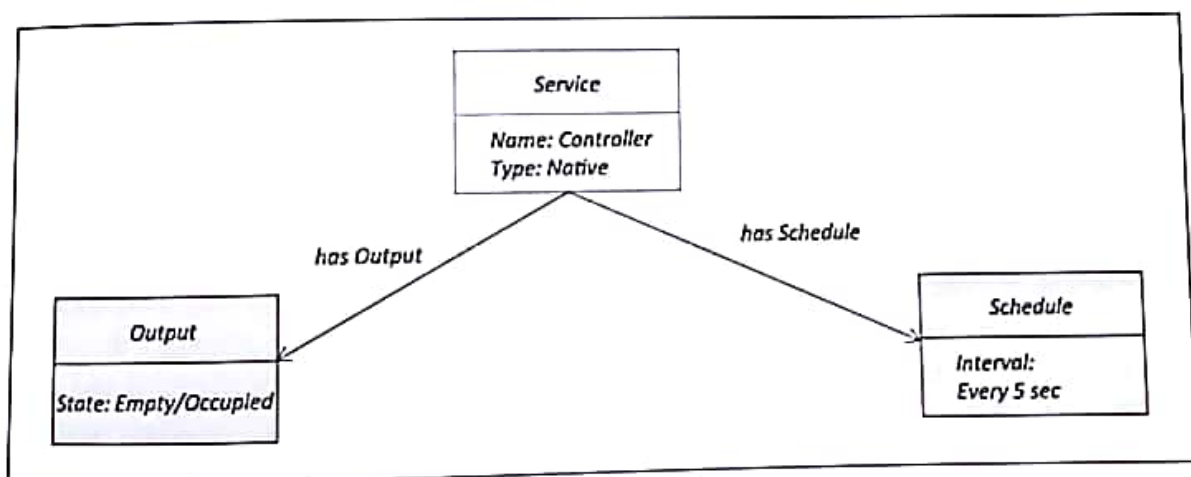


Figure 9.21: Controller service of the smart parking IoT system

■ Box 9.14: Serializer for REST service - serializers.py

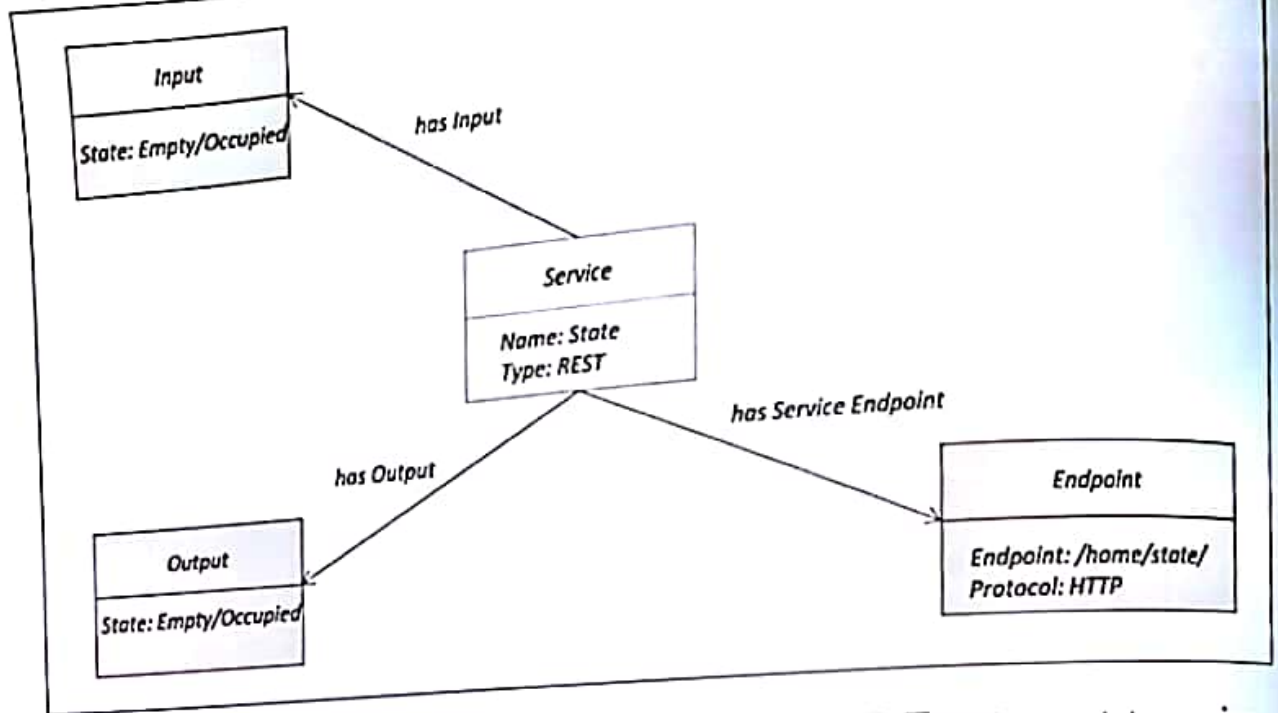


Figure 9.22: Service specification for the smart parking IoT system - state service

```

from myapp.models import State
from rest_framework import serializers

class StateSerializer(serializers.HyperlinkedModelSerializer):
    class Meta:
        model = State
        fields = ('url', 'name')
  
```

Box 9.15 shows the Django views for REST services and smart parking application. The ViewSets for the model (StateViewSet) are included in the views file. The *home* view renders the content for the smart parking application home page that displays the status of the parking slots. Notice that a request is sent to the state REST service to obtain the state of a parking slot. The code shown in this example is for a trivial case of a one-slot parking.

■ **Box 9.15: Django views for REST service and smart parking application - views.py**

```

from myapp.models import State
from rest_framework import viewsets
from django.shortcuts import render_to_response
from django.template import RequestContext
  
```

```
while True:  
    runController()  
    time.sleep(1)
```

9.4 Environment

9.4.1 Weather Monitoring System

REST-based Implementation

A design of a weather monitoring IoT system was described in Chapter-5 using the IoT design methodology. A concrete implementation of the system based on Django framework is described in this section. The purpose of the weather monitoring system is to collect data on environmental conditions such as temperature, pressure, humidity and light in an area using multiple end nodes. The end nodes send the data to the cloud where the data is aggregated and analyzed.

Figure 9.26 shows the deployment design for the system. The system consists of multiple nodes placed in different locations for monitoring temperature, humidity and pressure in an area. The end nodes are equipped with various sensors (such as temperature, pressure, humidity and light). The end nodes send the data to the cloud and the data is stored in a cloud database. The analysis of data is done in the cloud to aggregate the data and make predictions. A cloud-based application is used for visualizing the data. The centralized controller can send control commands to the end nodes, for example, to configure the monitoring interval on the end nodes.

Figure 9.27 shows a schematic diagram of the weather monitoring system. The devices and components used in this example are Raspberry Pi mini computer, temperature and humidity sensor (DHT22), pressure and temperature sensor (BMP085) and LDR sensor. An analog-to-digital (A/D) converter (MCP3008) is used for converting the analog input from LDR to digital.

Figure 9.28 shows the specification of the controller service for the weather monitoring system. The controller service runs as a native service on the device and monitors temperature, pressure, humidity and light every 10 seconds. The controller service calls the REST service to store these measurements in the cloud. This example uses the Xively Platform-as-a-Service for storing data. In the *setupController* function, new Xively datastreams are created for temperature, pressure, humidity and light data. The *runController* function is called every 10 seconds and the sensor readings are obtained.

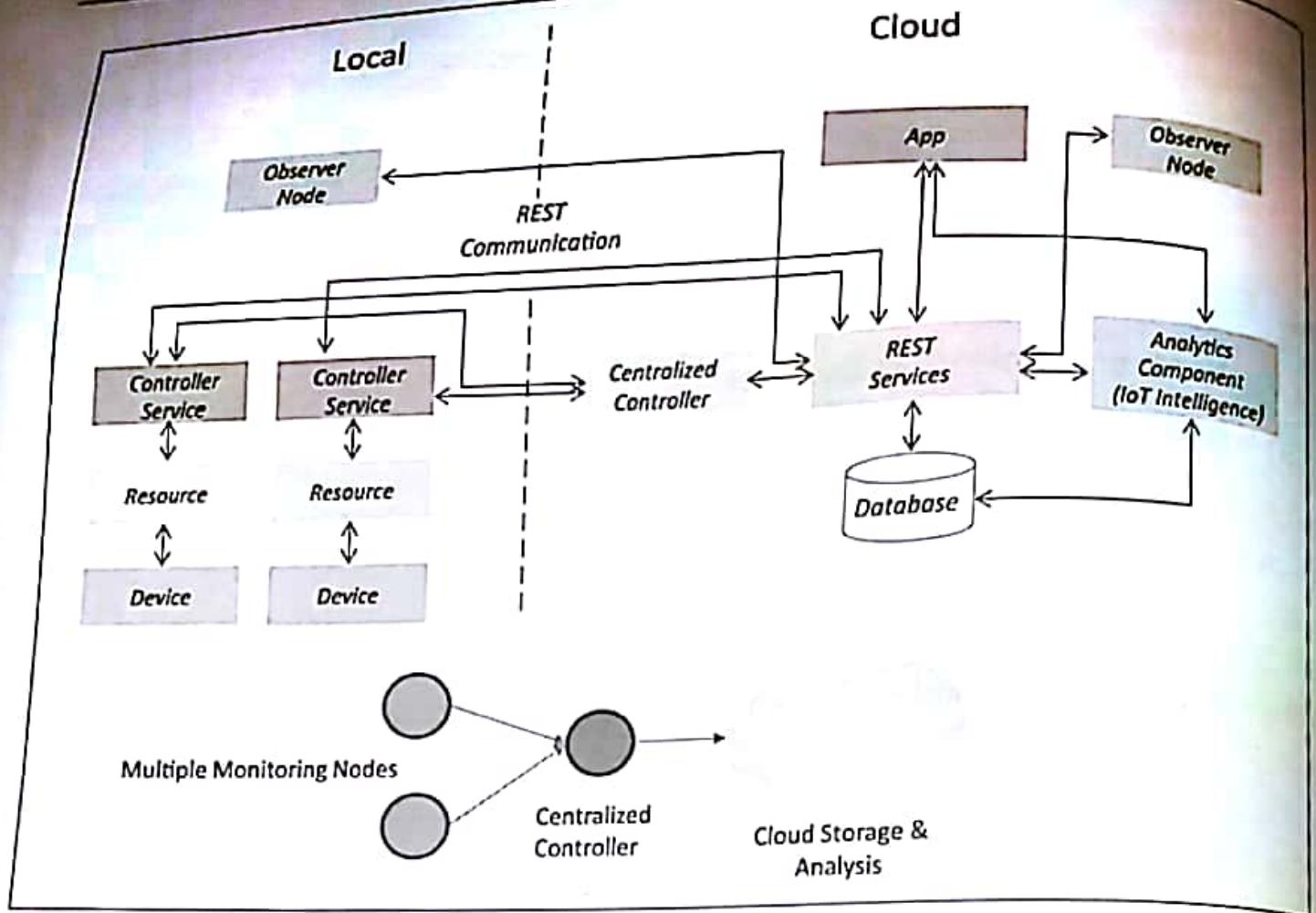


Figure 9.26: Deployment design of the weather monitoring IoT system

■ Box 9.19: Python code for controller native service - controller.py

```
import time
import datetime
import sys
import json
import requests
import xively
import subprocess
from random import randint
import dhtreader
from Adafruit_BMP085 import BMP085
import spidev
```

```
global temp_datastream
global pressure_datastream
```

```

pressure=pressure
humidity=humidity_datastream.current_value
light=light_datastream.current_value
lat=feed.location.lat
lon=feed.location.lon

return render_to_response('index.html', {'temperature': temperature,
'pressure': pressure, 'humidity': humidity,
'light': light, 'lat': lat, 'lon': lon,
context_instance=RequestContext(request)})

```

Figure 9.29 shows a screenshot of the weather monitoring web application.

WebSocket-based Implementation

The previous section described a REST-based implementation of the weather monitoring IoT system. In this section you will learn about an alternative implementation of the weather monitoring IoT system based on WebSocket.

The WebSocket implementation is based the Web Application Messaging Protocol (WAMP) which is a sub-protocol of WebSocket. You learned about Autobahn, an open source implementation of WAMP in Chapter 8. The deployment design for the WebSocket implementation is the same as shown in Figure 9.26.

Figure 9.30 shows the communication between various components of the WebSocket implementation. The controller in the WebSocket implementation is a WAMP application component that runs over a WebSocket transport client on the IoT device. WAMP application router runs on a WebSocket transport server on the server instance in the cloud. The role of the Client on the device in this example is that of a Publisher, while the role of the Router is that of a Broker. Publisher publishes messages to the topics managed by the Broker. Subscribers subscribe to topics they are interested in with Brokers. Brokers route events

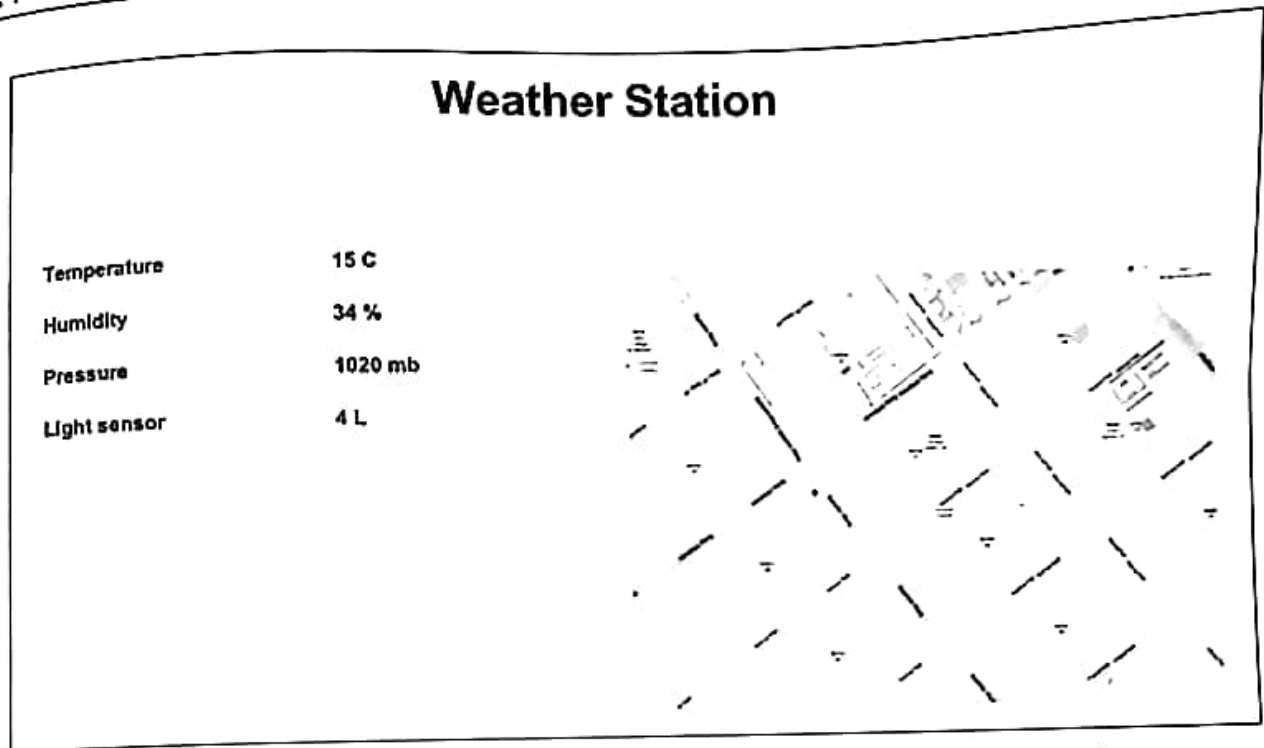


Figure 9.29: Screenshot of weather monitoring web application

incoming from Publishers to Subscribers that are subscribed to respective topics. Brokers decouple the Publisher and Subscriber. In this example, the Publisher and Subscriber run application code. The Publisher application component is the controller component, the source code of which is shown in Box 9.22. The Subscriber application component is the web frontend, the source code of which is shown in Box 9.23. The analytics component runs on a separate instance and subscribes to the topics managed by the Broker. Box 9.25 shows the code for a dummy analytics component. The communication between Publisher - Broker and Broker - Subscribers happens over a WAMP-WebSocket session.

■ Box 9.22: Controller code for WebSocket implementation - weathercontroller.py

```
from twisted.internet import reactor
from twisted.internet.defer import inlineCallbacks
from autobahn.twisted.util import sleep
from autobahn.twisted.wamp import ApplicationSession
import dhtreader
from Adafruit_BMP085 import BMP085
import spidev
```

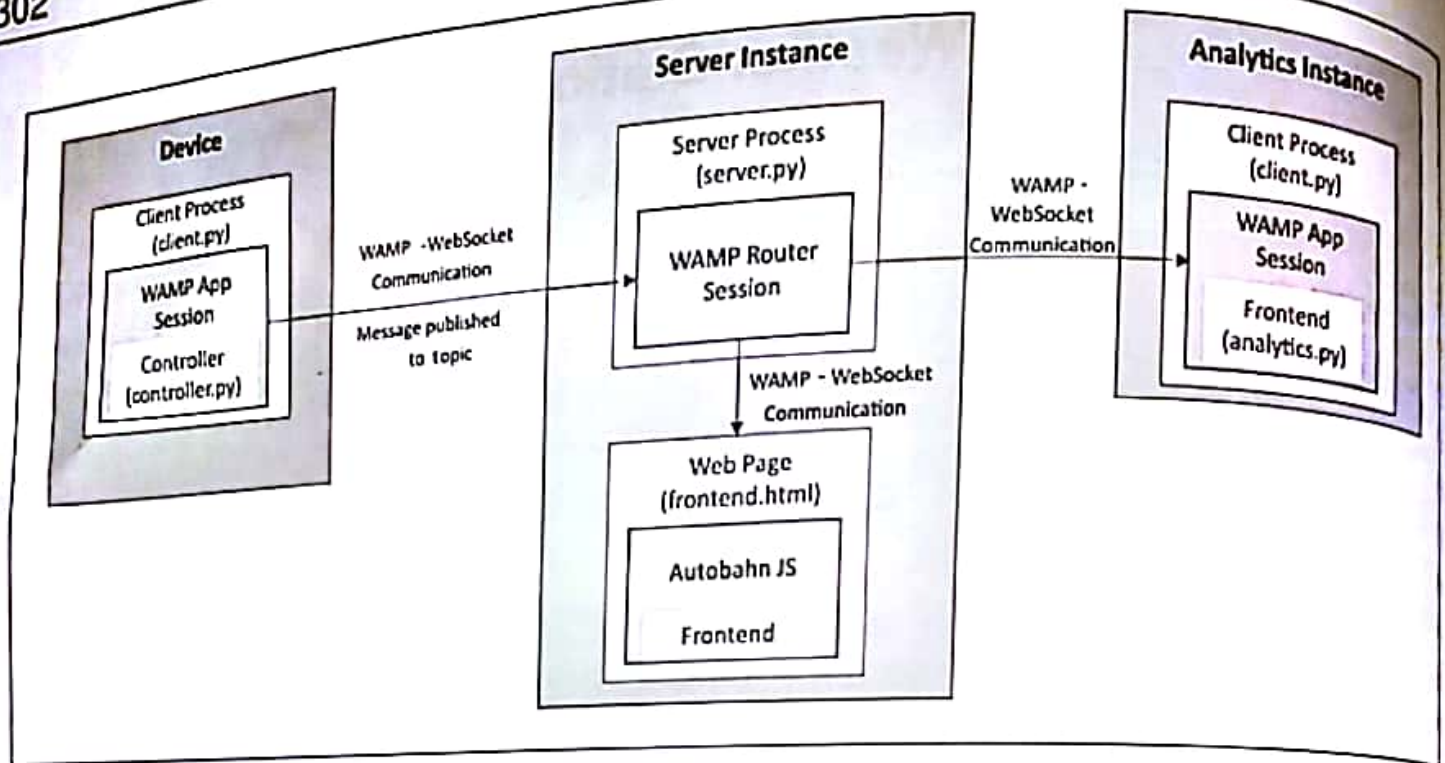


Figure 9.30: Components in the Websocket implementation

```

#Initialize DHT22 Temperature/Humidity Sensor
dev_type = 22
dht_pin = 24
dhtreader.init()

# Initialise BMP085 Temperature/Pressure Sensor
bmp = BMP085(0x77)

#LDR channel on MCP3008
light_channel = 0

# Open SPI bus
spi = spidev.SpiDev()
spi.open(0,0)

# Function to read SPI data from MCP3008 chip
def readChannel(channel):
    data = spi.xfer2([1, (8+channel)<<4,0])
    adc = ((adc[1]&3) << 8) + adc[2]
    return data

# Function to

```

```

from twisted.internet import reactor
from twisted.internet.defer import inlineCallbacks
from autobahn.twisted.wamp import ApplicationSession

#Placeholder for analysis function
def analyzeData(data):
    return True

#An application component that subscribes and receives events
class Component(ApplicationSession):
    @inlineCallbacks
    def onJoin(self, details):

        self.received = 0

    def on_event(data):
        print "Temperature: " + data[0] + "; Humidity: " + data[1]
        + "; Pressure: " + data[2] + "; Light: " + data[3]

#Placeholder for analysis function
analyzeData(data)

yield self.subscribe(on_event, 'com.myapp.topic1')

def onDisconnect(self):
    reactor.stop()

```

9.4.2 Weather Reporting Bot

This case study is about a weather reporting bot which reports weather information by sending tweets on Twitter. Figure 9.31 shows a schematic of the weather monitoring end-node. The end-node is comprised of a Raspberry Pi mini-computer, temperature, humidity, pressure and light sensors. In addition to the sensors, a USB webcam is also attached to the device.

Box 9.26 shows the Python code for the controller service that runs on the end-node. The controller service obtains the temperature, humidity, pressure and light readings from the sensors, every 30 minutes. At the same time an image is captured from the webcam attached to the device. The sensor readings and the captured image is then sent as a tweet on Twitter. To send tweets the controller service uses a Python library for Twitter called *tweepy*.

Bahga & Madiseti, © 2015

With tweepy you can use the Twitter REST API to send tweets. Before using the Twitter API, you would need to setup a Twitter developer account and then create a new application (with read-write permissions). Upon creating the application you will get the API key, API secret and access tokens. These credentials and tokens are used in the controller service.

■ **Box 9.26: Python code for weather reporting bot that tweets weather updates to Twitter**

```
import time
import datetime
import sys
from random import randint
import dhtreader
from Adafruit_BMP085 import BMP085
import spidev
from SimpleCV import Camera
from time import sleep
import tweepy

#Initialize USB webcam
myCamera = Camera(prop_set={'width':320, 'height': 240})

#Twitter Application Credentials
CONSUMER_KEY = "<enter>"
CONSUMER_SECRET = "<enter>"
ACCESS_KEY = "<enter>"
ACCESS_SECRET = "<enter>"

auth = tweepy.OAuthHandler(CONSUMER_KEY, CONSUMER_SECRET)
auth.set_access_token(ACCESS_KEY, ACCESS_SECRET)
api = tweepy.API(auth)

#Initialize DHT22 Temperature/Humidity Sensor
dev_type = 22
dht_pin = 24
dhtreader.init()

# Initialise BMP085 Temperature/Pressure Sensor
bmp = BMP085(0x77)

#LDR channel on MCP3008
```

```

date_time=date.strftime("%Y-%m-%d %H:%M:%S", time.localtime())
status = "Temperature: " + temperature + ", Humidity: " +
humidity + ", Pressure: " + pressure + ", Light: " + light

photo_path = '/home/pi/weather.jpg'

#Tweet weather information with photo
api.update_with_media(photo_path, status=status)

setupController()
while True:
    runController()
    time.sleep(1800)

```

Figure 9.32 shows a screenshot of a tweet sent by the weather reporting bot on Twitter.

9.4.3 Air Pollution Monitoring

IoT based air pollution monitoring systems can monitor emission of harmful gases by factories and automobiles using gaseous and meteorological sensors. This section provides an implementation of an air pollution monitoring IoT system. The deployment design for the system is similar to the deployment shown in Figure 9.26. The system design steps are similar to the weather monitoring system described in the previous section. Therefore only the schematic design and the controller implementation is provided.

The system consists of multiple nodes placed in different locations for monitoring air pollution in an area. The end nodes are equipped with CO and NO_2 sensors. The end nodes send the data to the cloud and the data is stored in a cloud database. A cloud-based application is used for visualizing the data.

Figure 9.33 shows a schematic diagram of air pollution monitoring end-node. The end node includes a Raspberry Pi mini-computer, MICS-2710 NO_2 sensor and MICS-5525 CO

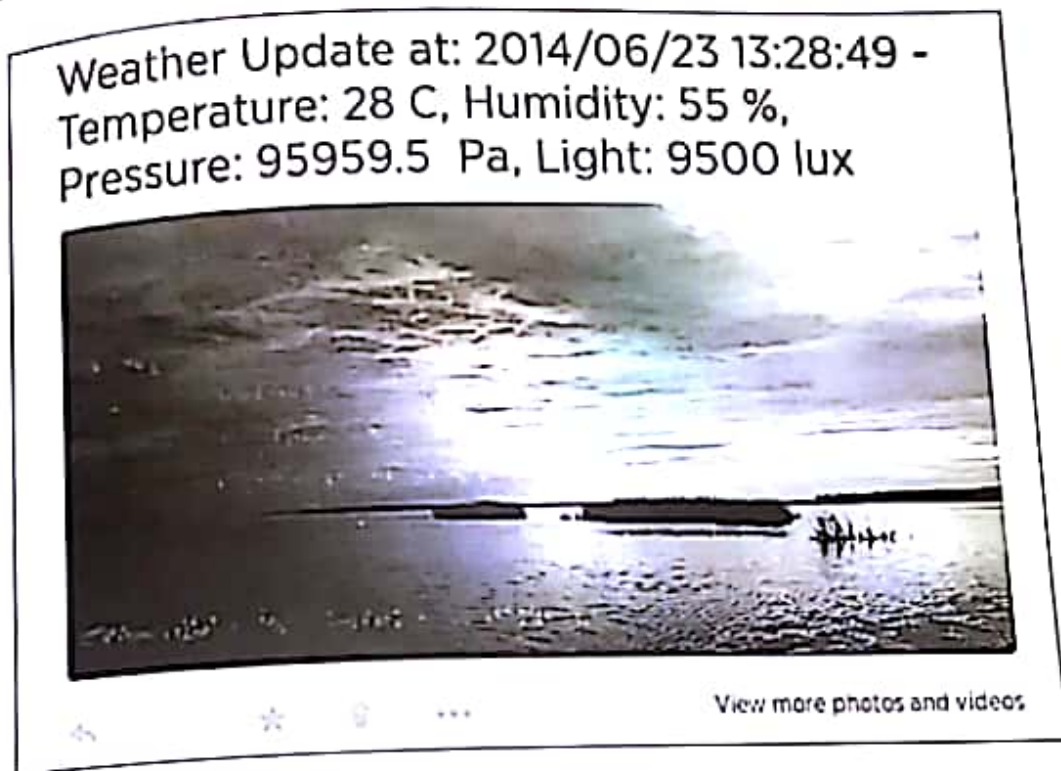


Figure 9.32: Screenshot of a weather update tweeted by the weather reporting bot

sensor. An A/D converter (MCP3008) is used for converting the analog inputs from the sensors to digital.

Box 9.27 shows the implementation of the native controller service for air pollution monitoring system. The controller service runs as a native service on the device and obtains the sensor readings every 10 seconds. The controller service calls the Xively REST service to store these measurements in the cloud.

■ **Box 9.27: Python code for controller native service - air pollution monitoring system**

```
import time
import datetime
import sys
import json
import requests
import xively
import spidev
```

```

except:
    datastream = get_COdatastream(feed)
    return datastream

#Controller setup function
def setupController():
    global NO2_datastream
    global CO_datastream

    feed = api.feeds.get(FEED_ID)

    feed.location.lat="30.733315"
    feed.location.lon="76.779418"
    feed.tags="Pollution"
    feed.update()

    NO2_datastream = get_NO2datastream(feed)
    NO2_datastream.max_value = None
    NO2_datastream.min_value = None

    CO_datastream = get_COdatastream(feed)
    CO_datastream.max_value = None
    CO_datastream.min_value = None

setupController()
while True:
    runController()
    time.sleep(10)

```

9.4.4 Forest Fire Detection

IoT based forest fire detection systems use a number of monitoring nodes deployed at different locations in a forest. Each monitoring node collects measurements on ambient conditions (such as temperature and humidity) to predict whether a fire has broken out.

An implementation of a forest fire detection system is described in this section. The system is based on a level-5 IoT deployment with multiple end nodes and one coordinator node. The end nodes perform sensing and the coordinator node collects data from the end

nodes and sends to the cloud.

Figure 9.34 shows a schematic diagram of forest fire detection end-node. The end node includes a Raspberry Pi mini-computer and DHT-22 temperature and humidity sensor. An XBee module is used for wireless communication between the end-node and the coordinator node. Figure 9.35 shows a schematic diagram of the coordinator node.

Boxes 9.28 and 9.29 show the implementations of the native controller services for the end node and coordinator node respectively. The controller service on the end node obtains the sensor readings every 10 seconds and writes the data to the XBee module which sends the data to the coordinator node. The controller service on the coordinator node receives the data from all end nodes and calls the Xively REST service to store these measurements in the cloud.

The XBee modules can be configured to communicate with each other using a Windows based application called X-CTU [127]. All XBee modules should have the same network ID and channel. The XBee module for the coordinator node has to be configured as a coordinator and the rest of the modules have to be configured as end devices or routers.

■ Box 9.28: Python code for controller service on end-node - forest fire detection system

```
import time
import datetime
import serial
import dhtreader

#Set Router ID
RouteID='123'

#Initialize DHT22 Temperature/Humidity Sensor
dev_type = 22
dht_pin = 24
dhtreader.init()

#Function to read temperature & humidity from DHT22 sensor
def read_DHT22_Sensor():
    temp, humidity = dhtreader.read(dev_type, dht_pin)
    return temp, humidity

def write_xbee(data):
    xbee=serial.Serial('/dev/ttyAMA0',baudrate='9600',
```

```

    datastream = feed.datastreams.create(
        "humidity"+str(id), tags="humidity")
    return datastream

#Controller setup function
def setupController():
    global temp_datastream
    global humidity_datastream

    feed = api.feeds.get(FEED_ID)

    feed.tags="Weather"
    feed.update()

    for i in range(1,numRouters+1):
        temp_datastream[i] = get_tempdatastream(feed,i)
        temp_datastream[i].max_value = None
        temp_datastream[i].min_value = None

        humidity_datastream[i] = get_humiditydatastream(feed,i)
        humidity_datastream[i].max_value = None
        humidity_datastream[i].min_value = None

setupController()
while True:
    runController()
    time.sleep(10)

```

9.5 Agriculture

9.5.1 Smart Irrigation

Smart irrigation systems use IoT devices and soil moisture sensors to determine the amount of moisture in the soil and release the flow of water through the irrigation pipes only when the moisture levels go below a predefined threshold. Data on the moisture levels is also collected in the cloud where it is analyzed to plan watering schedules.

An implementation of a smart irrigation system is described in this section. The deployment design for the system is similar to the deployment shown in Figure 9.26. The system consists of multiple nodes placed in different locations for monitoring soil moisture in a field. The end nodes send the data to the cloud and the data is stored in a cloud database. A cloud-based application is used for visualizing the data. Figure 9.36 shows a

schematic diagram of smart irrigation system end-node. The end node includes a Raspberry Pi mini-computer and soil moisture sensor. A solenoid valve is used to control the flow of water through the irrigation pipe. When the moisture level goes below a threshold, the valve is opened to release water. Box 9.30 shows the Python code for the controller native service for the smart irrigation system.

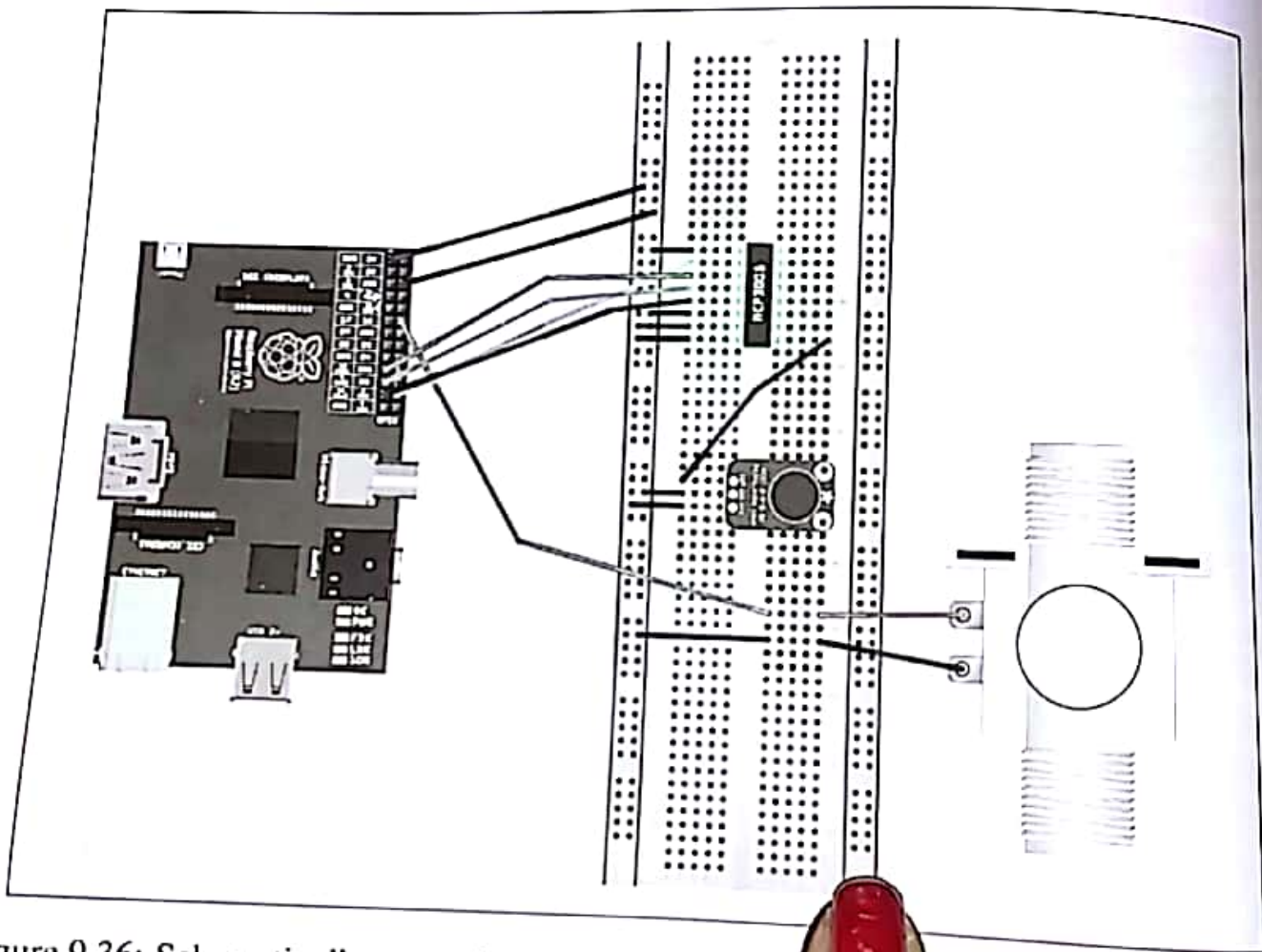


Figure 9.36: Schematic diagram of a smart irrigation system end-node showing the device and sensor

■ **Box 9.30: Python code for controller native service for smart irrigation system**

```
import time
import datetime
import sys
```

9.6.1 IoT Printer

the user's security

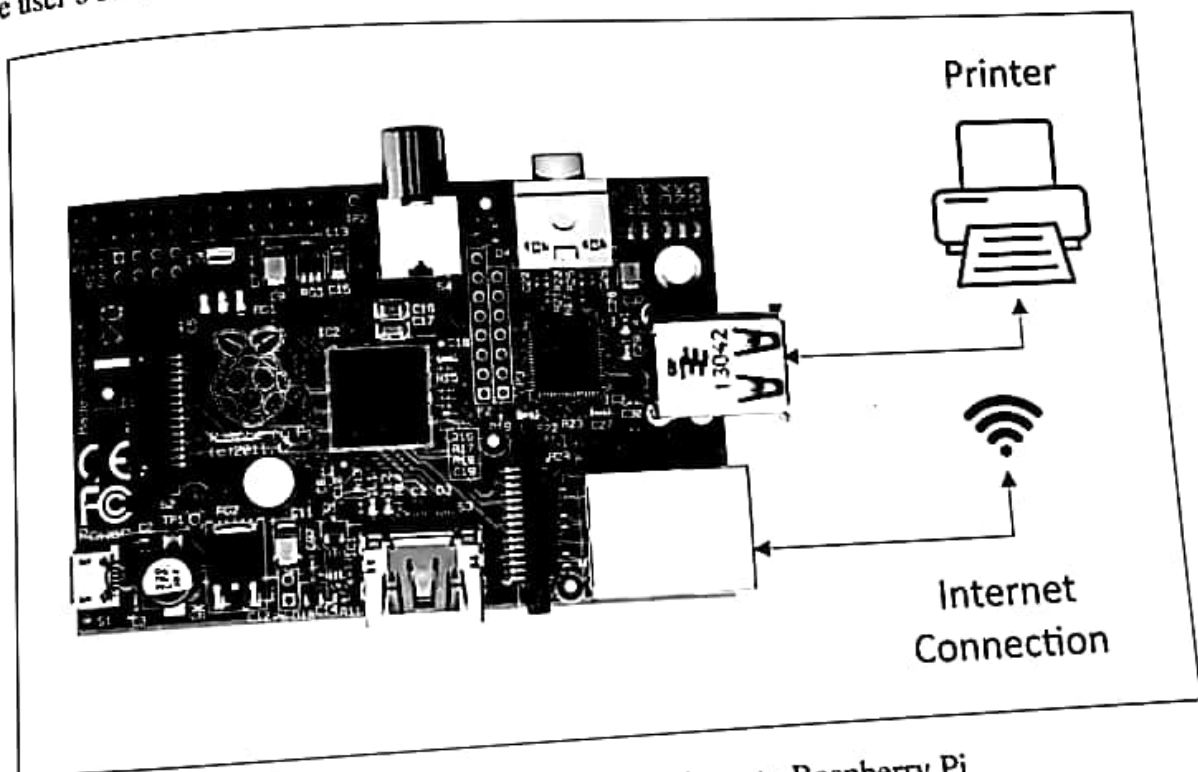


Figure 9.37: Connecting a printer to Raspberry Pi

Box 9.31 shows the code for the service that runs on the mini-computer which is connected to the printer.

■ **Box 9.31: Python code for IoT printer**

```
import gflags
import httplib2
import datetime
from apiclient.discovery import build
from oauth2client.file import Storage
from oauth2client.client import OAuth2WebServerFlow
from oauth2client.tools import run
```